Scan the Skill, Govern the Action: Composing Registry Verdicts with Runtime Consequence Control

Rohit Taneja
Pheo Inc
rohit@pheo.ai

Travis Weber
Pheo Inc
travis@pheo.ai

Abstract

Agent skill registries screen what they publish. OpenClaw's own security team showed that this screening is three disagreeing scanners under one judge rather than a single signal: the scanners overlap on at most 10.4% of combined positives, and 81.9% of flagged skills are caught by one scanner alone [1]. We take that finding as given and argue the open question is not *which scanner is right* but *which question is being asked*. Every signal in that pipeline answers "is this skill malicious?" None of them answers "is this action permitted on this machine, by this operator, right now?", and that second question has a different answer on the same artifact depending on whose infrastructure it runs on.

We report three measurements over 66,192 public ClawHub skill versions and a live agent. First, 789 skills across 174 unrelated publishers that all three scanners and the registry's own judge rate clean nonetheless instruct an action many operators forbid outright. 657 of them fetch code over the network and execute it in the same command. A manual audit puts our detector's precision at 73.6%, so roughly 581 are true positives. The scanners are not wrong and the code is not malware: permission and maliciousness are different predicates over the same artifact. Second, of 93 commands a live agent actually executed while following real skill documentation (40 skills, one agent), **3 appear verbatim in that documentation**. Static analysis reasons about a document that is empirically not the artifact that runs. Third, we set out what each layer structurally cannot see, including our own: a gate that reads actions is blind to harm that never becomes one. These blind spots do not coincide, which is why the layers compose rather than compete.

We contribute one design for the missing control point: a deterministic resolver (1.03 μs mean, 3.27 μs worst case, no model in the decision path) feeding a per-(resource, class) trust ledger whose promotion thresholds are derived from an operator's own stated risk tolerance rather than asserted. The common flat rule of "ten clean approvals" licenses a 25.9% failure rate at 95% confidence, up to 60$\times$ more permissive than the operator's own policy would allow. We argue agent-skill safety is a composition of two different questions, not a contest between two detectors, and we report where our own layer fails as precisely as where it helps.

1 Introduction

No bank checks a customer once. Know-your-customer runs at account opening: identity verified, source of funds established, sanctions lists cleared. It is a real control and a necessary one. Then the account opens, and a second control takes over that nobody would consider optional: every transfer afterwards is screened as it happens, against limits, against patterns, against what this account has done before. No regulator would accept "we verified them at onboarding" as a complete answer, because the customer who was verified on day one is not a promise about the transfer attempted on day four hundred.

Agent skills today have the first control and not the second. Registries screen a skill before it is published, thoroughly, with several independent instruments, and then an autonomous agent installs it and begins acting on a real machine with real credentials, and no control asks whether any particular action is one this organisation permits. This paper is about the second half of that pattern: what it costs to be missing, and what it takes to build.

That a second control is required is settled rather than open, and a commercial tier has formed around it (§10). We take that as established. Our question is narrower: given that something must sit in front of every action, on what principled basis does it ever stop asking? A control that interrupts an operator on every consequential action converges on being switched off, and an operator who switches it off is less protected than one who never installed it.

An agent skill is a directory containing a `SKILL.md`: natural-language metadata describing when a skill applies, and instructions an agent follows once it does. ClawHub is the public registry through which most OpenClaw skills are distributed. Before a skill version is published, it passes a pre-catalog verification gate: independent scans from VirusTotal, in-house static analysis, and NVIDIA SkillSpector are handed as context to ClawScan, an LLM-as-judge (GPT-5.5) that weighs them alongside provenance, metadata, and moderation history and issues one verdict: Clean, Suspicious, or Malicious [3].

In May 2026 OpenClaw published a measurement of that pipeline which we treat as this paper's premise rather than a finding to reproduce [1]. Across 67,453 skill versions, their three underlying scanners agree with each other almost never: any pair overlaps on at most 10.4% of combined positives, only 0.69% of flagged skills are caught by all three, and 81.9% of positives come from one scanner acting alone. Their diagnosis is precise, and we adopt it: *"we do not believe these findings point toward an issue with any of the individual scanners. Rather, each scanner has a different risk surface."*

1.1 The question nobody is asking

That diagnosis is usually read as a detection problem: four instruments, four calibrations, unify them and the noise resolves. We think it is something else. VirusTotal answers "is this a known-bad binary." Static analysis answers "does this match a dangerous pattern." SkillSpector answers "is this capability profile over-broad." ClawScan, the judge over the three, answers "on balance, is this malicious." Four different questions, all of them about a property of the *artifact*.

Consider a skill published by `oomol`, a real company, whose documentation instructs:

```
curl -fsSL https://cli.oomol.com/install.sh | bash
```

Every signal in the pipeline calls this clean, and each is correct: it is an ordinary vendor installer, not malware. Yet a large class of operators, anyone under a control framework that forbids executing unreviewed remote code (which includes most regulated infrastructure), will not permit an autonomous agent to run it unattended, from any publisher, malicious or not. The prohibition is a property of the *action*, evaluated against *this operator's policy*, not a property of the artifact.

No scanner in the pipeline can express that. Their entire output vocabulary is clean / suspicious / malicious. There is no slot for "acceptable at a two-person startup, prohibited at a bank," because that judgment depends on facts, whose machine, whose data, whose compliance regime, what this skill has done here before, that do not exist at publish time and are not properties of the file.

1.2 Contributions

1. **Permission ≠ maliciousness, measured.** 789 skills across 174 unrelated publishers that all three scanners and the judge rate clean while instructing an action a large class of operators forbids outright (§4).

2. **The scanned artifact is not the executed artifact.** Of 93 commands a live agent issued while following real skill documentation, 3 (3.2%) appear verbatim in that documentation (§5). Agent tool-use logging is well established; what we have not found measured elsewhere is the gap between what agent-skill static analysis inspects and what the agent then executes.

3. **A live demonstration.** A real agent, real skills the registry cleared with quoted high-confidence rationales, a real gate, blocked before execution (§6).

4. **A layer-by-layer account of what each control can and cannot see**, including our own blind spot, harm that never resolves to an action, stated as plainly as our contributions (§7).

5. **Derived, not asserted, autonomy thresholds**, parameterised by the operator's own risk tolerance, with a measured resolver fast enough to sit on every tool call (§2.1, §8).

6. **A benchmark for action-level governance**, released with the paper: 64 semantics-preserving rewrites across nine techniques, against which any implementation can report a resolution rate. Ours is 77% (§2.2). The field has no shared way to state how much of the equivalence class a gate covers, so no two gates can be compared and no implementation can show it is improving.

2 OATS: Resolving Actions, Not Judging Artifacts

OATS (Open Agent Trust System) occupies the control point none of these signals sit on: the moment an agent has decided what to do and has not yet done it. It does not read the skill's prose, and it does not render an opinion about the skill. It answers one question about one concrete act: *is an agent with this track record permitted to perform this class of action on this resource, unattended, under this operator's policy?*

2.1 The deterministic resolver

Let Σ^* be the space of command strings an agent may emit and $\mathcal{C}$ a finite, closed set of consequence classes. $\mathcal{C}$ has 33 members, derived from the operations the governed surfaces expose rather than from a taxonomy chosen in advance: shell execution and its three dangerous specialisations (remote execution, credential access, destructive); file writes split by what the path implies (docs, tests, boilerplate, business logic, dependencies); repository and CI operations (branch merges, protected merges, branch protection, workflow edits and runs, releases, webhooks, repository deletion); secret and IAM changes; deploys; and the computer-use verbs (observe, interact, type, commit). Twelve of the 33 never graduate (§8). The resolver is a total function

$$\rho : \Sigma^* \to \mathcal{C}$$

with three properties we hold it to. It is *total*: every command receives a class, so no input path fails open. An early defect in which unmatched shell commands fell through as unclassified is reported in §9. It is *deterministic*: ρ is a pure function of the command string, so $\rho(c)$ is identical across runs, hosts, and releases, and is reproducible by an auditor without re-executing a model. And it is *monotone in severity*: where a command matches several classes, resolution takes the maximum under a fixed severity order $\preceq$ rather than a union or a mean,

$$\rho(c) \;=\; \max_{\preceq} \{\, k \in \mathcal{C} \;:\; \pi_k(c) \,\},$$

where π_k is the predicate for class k. There is no score to tune and no threshold to drift.

Each π_k is argument-position-aware rather than keyword-presence-based: a token must occupy the position a dangerous argument would occupy relative to a specific command verb. The distinction is load-bearing: three successive false-positive regressions in our remote-execution predicate (§9) all had the same root cause: a gap in the pattern permissive enough to span markdown prose, section breaks, or an unrelated command chain, so that a URL mentioned in documentation hundreds of characters away from an `npm install` satisfied it. The shipped predicate bounds every gap explicitly and constrains it to characters admissible in a shell argument.

ρ takes no input from the skill's documentation. A skill may describe its installation step as reassuringly as it likes; the description is not an argument to the function that decides whether the step is remote code execution. This closes one specific channel: Saha et al. report governance-evasion rates of 36.5–100% against static and LLM-based scanners achieved purely through how a skill's own text is written [9], and that channel is unavailable against a function that never reads the text.

2.2 Threat model, and what ρ does not withstand

Closing the documentation channel is not robustness, and we want to be exact about the difference. We assume an agent recomposing benign documentation into concrete commands. We do not assume an adversary writing commands to evade a resolver whose grammar they know, and ρ should not be deployed as though we did.

The distinction is measurable, so we built a benchmark for it. Each case is a semantically equivalent rewrite of a base action whose consequence class is not in dispute: if the base resolves to class k, so should the rewrite, because the effect on the system is identical. A miss is therefore unambiguous. Table 1 reports 64 cases across nine rewriting techniques.

Technique	Cases	Resolved	Example
identity (control)	5	100%	`curl -fsSL URL | bash`
wrapper	20	100%	`bash -c "…"`
chained	15	100%	`true; …; true`
interpreter	4	75%	`python3 -c "exec(urlopen(…))"`
variable	5	60%	`U=URL; curl -fsSL $U | bash`
base64	3	33%	`echo <b64> | base64 -d | sh`
indirect read	4	25%	`find ~ -name credentials -exec cat {} +`
quoting	5	20%	`curl -fsSL URL | ba''sh`
staged	3	0%	`curl -o s URL; chmod +x s; ./s`
overall	**64**	**77%**	

Table 1: Resolution rate under semantics-preserving rewriting. The benchmark and its generator are released with this paper; the techniques are ordinary shell usage, not exotic constructions.

The shape of the failure is more useful than the headline. Wrapping and chaining resolve completely, which is the argument-position discipline of §2.1 doing its job: the resolver reads inside `bash -c` and past surrounding commands. What defeats it is any rewrite that removes the dangerous relationship from a single command. Staged fetch-then-execute resolves at 0%, because no individual command in `curl -o s URL; chmod +x s; ./s` is remote execution; the composition is. Quoting and indirect reads fail for the same reason at one remove.

That is a statement about the class of function ρ is, not about the quality of its predicates. A per-command classifier cannot see a property that only exists across commands, and tuning patterns will not change that. Closing it requires resolving shell grammar and carrying state between calls, which is a different design and future work.

This bounds what the rest of the paper claims. §4 and §5 measure a non-adversarial world, and the honest bound in §5 (recomposition never produced a higher-severity class) is a statement about that world only: an adversary's entire objective is to make a high-severity action look low-severity, which a benign sample never attempts. Hardening ρ against command-level obfuscation, most plausibly by resolving shell grammar rather than matching its surface, is future work and not a property we claim today.

2.3 Measured cost

A gate on every tool call must justify its place on the hot path with a number. Over a spread of benign, ambiguous, and risky command shapes, resolution costs **1.03 μs** mean on one core of an Apple M-series laptop, 3.27 μs on the slowest shape (`curl …| bash`), or about 971,000 classifications per second. End to end the current CLI hook takes 67.6 ms median and 81.1 ms at p95, dominated by process spawn and local IPC rather than by classification: an engineering cost, not an architectural one.

The distinction matters for what is architecturally possible. A judge that consults a language model, local or hosted, pays a model's latency on every decision, which bounds where it can be placed. A resolver at 10^{-6} seconds can sit synchronously in front of every action without the operator perceiving it. We make no claim that ρ is smarter than an LLM judge; it is deliberately narrower (§7). We claim only that it is cheap enough, and stable enough, to occupy a position an LLM judge cannot.

2.4 The trust ledger

ρ's output feeds a finite-state ledger keyed on the pair (resource, class), not on the skill. Trust is per-lane: a skill may be fully trusted to write documentation in one repository and permanently untrusted to read a

credential file, at the same time. Each lane moves

$$\textit{unclassified} \to \textit{held} \to \begin{cases} \textit{approved} & (\text{clean-run counter } n \mathrel{+}= 1) \\ \textit{rejected} & (n \leftarrow 0) \end{cases}$$

with *graduated* reachable only when $n \geq N_k$, the per-class threshold derived in §8, and only for classes whose severity permits graduation at all. Every transition is a counting rule over recorded human decisions, replayable from the ledger alone.

3 Data and Method

We use `OpenClaw/clawhub-security-signals`, OpenClaw's own MIT-licensed dataset [2], in full: all four splits, 66,192 deduplicated skill versions, each carrying four scanner verdicts and redacted `SKILL.md` content. Every skill is classified through the unmodified shipping client (`oatsctl hook pre-tool-use`), the same binary an agent's tool-use hook invokes, rather than a reimplementation, for a reason documented in §9.

Throughout, we deduplicate findings by publisher. A single publisher repeating one installer line across dozens of their own skills is one finding, not dozens; the raw counts are 2–3× larger and we do not report them as headline numbers.

We also exclude one class, `secret_change`, from all risk counts. It fires correctly: a skill documenting a write to `.env` or `~/.config/…/credentials.json` genuinely is a secret operation. But nearly every skill requiring an API key does this, making it a correct classification and a weak risk signal. Including it would have inflated our central finding by roughly 2×.

4 Finding: Clean Skills Instructing Forbidden Actions

Of 66,192 skills, 29,257 are rated `clean` by all three scanners and by the ClawScan judge. Within that population, 789 skills across 174 distinct publishers instruct an action in a class many operators forbid unattended (Table 2).

We report this population rather than the larger one available. Relaxing the condition to *no signal returned suspicious or malicious* admits a further 1,574 skills whose scans did not complete: 731 with no SkillSpector result, 438 with no VirusTotal result, 393 with no static result, plus `stale`, `pending`, and `error` states. That set yields 822 skills across 187 publishers, but "nothing objected" there includes "nothing looked," which is a weaker claim than the one we want to make.

Action class instructed	Skills
Fetch and execute remote code	657
Credential access	87
Destructive command	26
IAM change	20
Deploy	3
Total (174 distinct publishers)	789

Table 2: Skills every scanner and the judge rated clean, instructing actions a large class of operators prohibit for autonomous agents. These are detections of policy-relevant capability, not of malice.

These 789 are flagged by our detector, not confirmed by hand. A manual audit put its precision at 73.6% (§9), so roughly 581 are true positives; the finding does not turn on the exact count. Nor are they missed attacks, and the scanners did not err: the publishers here include established vendors shipping ordinary installers. The finding is that *maliciousness and permission are different predicates over the same artifact*, and a pipeline computing only the first leaves the second unanswered for every skill it clears.

5 Finding: The Scanned Artifact Is Not the Executed Artifact

Static analysis of `SKILL.md` is sound only if what the agent executes is what the document contains. We tested that assumption directly. For 40 skills sampled across distinct publishers, we classified every fenced code block in the document (the static view) and separately captured every command a live agent (Claude Sonnet 5) issued while following that same document, classifying both through the same resolver.

Executed commands ($n = 93$, 40 skills)	Count	Share
Verbatim from `SKILL.md`	3	3.2%
Recomposed (class present in document)	56	60.2%
Class absent from every documented block	34	36.6%

Table 3: What an agent actually runs, against what the document says. 96.8% of executed commands do not appear in the artifact the scanners inspected.

Agents do not transcribe documentation; they recompose it. They chain a check onto an install the document never chained (`go version && go install …`), add flags, redirect output, split one documented step into three. In 42% of skills the agent performed an action whose consequence class appeared nowhere in the document's own code blocks.

An honest bound on this result. All 34 unanticipated commands resolved to the generic `shell_exec` class. We observed *no* case in this sample where recomposition manufactured a *higher-severity* class than the document contained. The claim this experiment supports is therefore precise and narrower than it may first appear: the executed string is essentially never the documented string, so document-based analysis reasons about a proxy. But in this sample, the proxy did not fail in a dangerous direction. We report the absence of that stronger result because a reader would otherwise assume it.

6 Experiment: Live Runtime Verification

We ran a live agent (Claude Sonnet 5) against real skill documentation through the unmodified gate, executing only commands the gate permitted. Each skill received one run of at most four turns, with its `SKILL.md` in the system prompt and a single instruction to install it and confirm it works; a refused command returned the gate's own refusal text as the tool result. The harness is in the repository named under Availability. Across 54 unanimously-clean skills that instruct a risky action, the agent reached for such an action in 4 of 54; all four were held or blocked before execution. We report both rates: the first shows how often a cleared skill drives an agent toward a forbidden action, the second whether the gate stops it.

Skill	ClawScan's own verdict (verbatim)	What the gate did
`minirr890112-byte/llm-deploy-helper`	clean, high confidence: "coherent and user-directed"	`pip install git+…`: denied, remote code execution
`rogeecn/memos-cli`	clean, high confidence: "a coherent Memos CLI helper"	`go install …@latest`: denied
`snuri00/osint-mcp`	clean, medium confidence: "does not show malicious behavior"	`pip install git+…`: denied
`charleshahn/duivytools-skills`	clean, high confidence: "no hidden or abusive behavior found"	ordinary install held for review; retry via a third-party mirror denied

Table 4: Four live blocks, each on a skill the registry's own judge had cleared, with its rationale quoted.

The fourth case shows the resolver reading an execution rather than a description. The agent's first install attempt was ordinary and was *held for routine review*, not blocked. Its second attempt routed the same install through a third-party mirror, and the same resolver classified that occurrence as remote code

execution and denied it. One documented step, two executions, two different classes, because ρ is a function of what actually happened, not of what the document said would happen.

7 What Each Layer Sees

The four registry signals and OATS are often discussed as if they were rival detectors with different accuracy. They are not comparable that way, because they do not take the same input, do not run at the same time, and do not emit the same kind of statement. Table 5 sets them side by side on those axes rather than on a shared score.

Layer	Question it answers	Input	When	What it cannot see
VirusTotal	Is this a known-bad binary?	Bundled files	Publish	Novel code; anything not in a signature database
Static analysis	Does this text match a dangerous pattern?	Document text	Publish	Intent; whether the pattern is ever reached
SkillSpector	Is this skill's declared capability profile overbroad?	Document text, AI-assisted	Publish	Which capabilities are actually exercised, and on what
ClawScan	On balance, is this skill malicious?	The three above, plus provenance and moderation history	Publish	Anything absent from the artifact at publish time
OATS	Is *this action* permitted here, now, given this record?	The resolved command string	Every action	Harm that never becomes an action

Table 5: Five layers, five different questions. The right-hand column is the point: each blind spot is a property of where the layer sits, not of how well it is built.

The last column describes where each layer sits, not how well it performs. VirusTotal is blind to novel code because reputation requires history. SkillSpector is blind to which of a skill's capabilities are actually used, because that is not a fact about the document. And OATS is blind to harm that never resolves to an action, because it only ever sees actions.

That last blind spot is real and we state it plainly with examples rather than leave it implied, quoting ClawScan's own summaries. `neuralshift1/gmail-last5` "hard-codes a specific Gmail account and lacks a clear user confirmation step" and has no executable block at all. `arubiku/mia-twitter-stealth` is "designed to automate Twitter/X account actions while hiding from bot detection" through individually unremarkable tool calls. `douglascorrea/obsidian-remote` "gives an agent very powerful access to private notes and app internals without enough safety boundaries," a scope-versus-purpose mismatch in which no single action is disproportionate. A hardcoded recipient, a persuaded behaviour, an over-broad scope: three shapes of harm that SkillSpector's Excessive Agency analysis exists to catch and that a function reading only a command string cannot see. A runtime action gate is not a superset of a semantic scanner and must not be deployed as one.

7.1 Why the blast radius is wider together

Because the blind spots do not coincide, the layers compose rather than compete. A skill can be malicious in the artifact and never act (caught upstream, invisible to us). It can be entirely benign in the artifact and still drive an action an operator forbids (invisible upstream, caught by us; §4 counts 789 such skills across 174 publishers). And an artifact that never changes can produce different actions on different runs, which is the case §5 measures at 96.8% and §6 exhibits concretely: the same documented install step resolved as ordinary on one attempt and as remote code execution on the retry.

Empirically the two signals behave as near-independent: across 66,192 skills, the Jaccard overlap between OATS's flags and the union of the registry's own signals' positives is 0.022. We report that as evidence of independence, not of superiority. Low overlap is what one expects from instruments reading different objects, and it is the reason the union covers a wider blast radius than either alone.

This suggests a composition rule cheaper and stronger than new detection: *a registry verdict is an input to the ledger, not a competitor to it.* Where an upstream evaluator has marked a skill non-clean, an operator's policy can pin every lane for that skill at held or never-graduates, regardless of what ρ concludes about any individual command. The scanner's artifact-level judgment and the gate's action-level enforcement compose without either approximating the other. We present this as a mechanism rather than a measured result: evaluating it against the population that the scanner verdict itself defines would be circular.

Two predicates we built and did not ship. While examining what OATS misses, we implemented two candidate detectors, outbound data sent to a literal hardcoded domain and a credential embedded directly as a command argument, and held them to the specificity bar every shipped predicate must pass: measured false-positive rate against the 29,257-skill unanimously-clean population. They fire on 4.4% and 1.5% of clean skills respectively, because `curl`-POSTing data to a URL is simply how ordinary API integrations work. Either would manufacture, inside OATS, the alert fatigue OpenClaw names as unsolved in their own approval system [4]. Neither shipped. The same false-positive bar that rejected these two is the one every shipped predicate had to clear.

When there is no verdict to compose with. Not every skill arrives through a registry. OpenClaw's own roadmap acknowledges skills coming from GitHub, a private registry, or a file someone sends [4]. For those no upstream signal exists at any price, and the runtime layer is the only one.

8 Graduated Autonomy as Operator Policy

A gate that never lets a track record reduce scrutiny recreates exactly the prompt fatigue OpenClaw reports in their own system: users enable unattended mode once prompts outpace reading, after which no prompt is read. A promotion rule has to be defensible, and "ten clean approvals" was chosen rather than derived.

By the rule of three, zero failures across N independent trials bounds the true failure rate ϵ at confidence $1-\delta$ via $(1-\epsilon)^N = \delta$, giving the threshold required for a class whose tolerable failure rate is ϵ_k:

$$N_k \;=\; \left\lceil \frac{\ln \delta}{\ln(1-\epsilon_k)} \right\rceil .$$

At $\delta = 0.05$, ten clean approvals bound the failure rate at only **25.9%**, a one-in-four chance the eleventh action is wrong.

Consequence class	ϵ_k	N_k
Docs, tests, reads	10%	29
Shell execution	5%	59
Business logic, dependency changes	2%	149
Deploy, IAM, CI configuration	0.5%	598
Remote exec, credentials, secrets, destructive	0%	never

Table 6: Clean runs required to license unattended action at 95% confidence. A flat threshold of 10 is 3–60× too permissive, and unsound at any N for the bottom row.

The rule of three assumes each approval is an independent trial. An agent's repeated actions on one resource are correlated, so N_k is best read as a floor: dependence among approvals can only raise the evidence a sound threshold requires, never lower it. A tighter bound would need a model of that dependence; the value of N_k is that it is the least threshold a practitioner can defend, not the exact one.

ϵ_k is the operator's parameter, and it is where "what this organisation permits" enters the system as a number rather than a vendor default. A two-person startup and a regulated bank do not owe each other the same tolerance on a deploy action, and the same derivation tells each what their own stated tolerance costs in evidence. Classes in the bottom row admit no finite N, since no clean record makes an unrecoverable action recoverable, which is how the policy of §4 becomes mechanical.

9 Limitations and Defects in Our Own Instrument

Silver-standard ground truth. OpenClaw's dataset card describes `clawscan_verdict` as the registry's automated verdict, not human-adjudicated truth, with a human-annotated subset still under development [2]. "Scanner-confirmed malicious" is not "verified malicious," and every statement we make about that population inherits the limitation.

Sample sizes. 54 skills for §6 and 40 for §5 demonstrate mechanisms and produce non-trivial rates; they do not support tight confidence intervals.

We do not report a detection rate for OATS against malicious skills, by choice. Such a number would score a runtime action gate on a publish-time malware-detection benchmark, which is not the question it answers (§7); a favourable score there would in fact indicate redundancy with the scanners rather than value beside them. We state the blind spot qualitatively and with named examples instead.

Precision. A manual audit of the static instruction-detection method returned 73.6% precision; roughly one in four flagged skills in §4 is likely a false positive, and we have no human-adjudicated labels for that population.

Single registry, frozen snapshot. One registry, one deliberately frozen snapshot chosen for reproducibility.

Six defects in our own instrument, found and fixed during this work. A shell-classification bypass let unmatched Bash commands fail open. Three successive false-positive regressions in the remote-execution predicate, each a too-permissive gap, were caught only by measuring against the clean population. A performance defect fetched 152 MB of room history per governance check, which would have made corpus-scale evaluation impractical and any busy production room unusable. An entropy heuristic classified relative file paths (`references/CLAUDE-local-template.md`, 4.21 bits) as leaked credentials.

Two of the six surfaced only when we ran the instrument across all 66,192 skills, which is itself the argument for corpus-scale evaluation.

10 Related Work

ρ is a reference monitor for agent actions, and the criteria are Anderson's: invoked on every access, tamper-proof, and small enough to verify [5]. ρ sits on every tool call, admits no model into the decision path and replays from the ledger, and is a pure function of one string. Saltzer and Schroeder's complete mediation and least privilege [6] are the same reasons the ledger is keyed on (resource, class) rather than on the skill. We claim no novelty in these principles; the contribution is that agent actions are a surface where they had not been applied, and §2.2 is candid that the third criterion is currently met by a syntactic matcher an adversary can route around.

Runtime guardrails for LLM applications are an active area. Llama Guard [7] classifies conversational input and output with a model; NeMo Guardrails [8] runs programmable rails over an application's dialogue flow. Both put a model or a scripted policy between a user and a model. OATS sits one layer lower, between a model's chosen action and the system it would change, and the difference that matters for deployment is cost: a rail consulting a model pays a model's latency per decision, which is what confines it to conversational turns rather than every tool call.

OpenClaw's scanner-disagreement study [1] is this paper's premise rather than a comparison point. Saha et al. demonstrate discovery- and selection-manipulation attacks against ClawHub's ranking and 36.5–100% governance evasion against static scanners [9]; their attacks target what a human or ranking algorithm sees before install, complementary to our focus on what an agent does after. Recent work on trajectory assurance argues individually permitted actions can jointly violate a state-conditioned invariant that no deployed mechanism enforces [10]; our per-(resource, class) ledger is one concrete instantiation of the enforcement point

that work argues is missing, though we enforce per-action rather than per-trajectory. Formal machinery for temporal constraints over agent action sequences [11] is a natural direction for extending ρ's policy language beyond the per-action scope we implement.

Runtime governance as an emerging tier. The argument that agent safety requires a runtime control point is no longer only an academic position. A commercial category has formed around it: venture-funded platforms now discover agents across enterprise estates, record their tool invocations, and intervene on anomalous behaviour in production, with published deployments spanning tens of thousands of agents at a single customer. We take this as independent confirmation of the premise, and claim no part of the runtime tier as our invention. It also sharpens what remains genuinely open. Across both the registry-side scanning described in §7 and this emerging runtime tier, controls are binary in the same way: an action is permitted, or it raises an alert and waits for a human. What no published system in either tier provides is a principled account of when scrutiny may be safely relaxed. Absent that, the operator's only recourse for reducing prompt volume is to disable the control, which is precisely the failure OpenClaw documents in their own system [4]. §8 derives how much evidence a control should require before it stops asking.

11 Conclusion

OpenClaw showed that agent-skill security cannot be one scanner's allow/block call. We agree, and we argue the reason is that the pipeline computes maliciousness while operators need permission, a different predicate, over an artifact the scanners never see: across 93 commands from 40 skills, 96.8% of what the agent executed did not appear in the document they inspect. Our contribution is a control point for that gap: a deterministic resolver cheap enough (1.03 μs) to sit on every action, a per-lane trust ledger, and promotion thresholds derived from the operator's own stated tolerance rather than a vendor's constant. We do not claim it is the only such design, and §2.2 bounds what this one withstands.

We are equally direct about the limit. A skill whose harm is a hardcoded recipient, an undisclosed scope, or a persuasive instruction produces no action for a gate to resolve, and no policy strictness changes that. It is exactly the surface a semantic scanner reads and we do not. Two candidate predicates that would have widened our reach failed our own false-positive bar and were not shipped. The measurements make the case for both layers together: scan the skill, and govern the action.

Availability

Every measurement here can be re-derived from public artifacts. The corpus is OpenClaw's own MIT-licensed dataset at `https://huggingface.co/datasets/OpenClaw/clawhub-security-signals`; the scanner-agreement figures in §7 are two SQL queries in its own browser console. The profile, this paper, and the script reproducing §4 are at `https://github.com/pheo-ai/open-agent-trust-system` under Apache 2.0. That script runs the corpus through a released implementation rather than a private build, so the result does not depend on access to ours:

```
pip install pheo-oats pandas pyarrow requests
oats start --no-browser &
python research/reproduce_clawhub.py
```

The implementation used for every measurement here is `pheo-oats` 0.5.3, published for macOS, Linux, and Windows at `https://pypi.org/project/pheo-oats/`. The resolver's decision path contains no model, so a rerun reproduces the same classes rather than approximating them. The live-agent experiments in §6 and §5 depend on a model's behaviour and will vary.

References

[1] V. Koc, P. Erichsen, J. Tomlinson, A. Rivera, M. Appel, N. Paz. *ClawHub Security Signals: When VirusTotal, Static Analysis, and SkillSpector Disagree*. arXiv:2606.01494, May 2026.

[2] OpenClaw. *ClawHub Security Signals* [Dataset]. https://huggingface.co/datasets/OpenClaw/clawhub-security-signals, 2026. MIT License.

[3] V. Koc, P. Erichsen. *OpenClaw Collaborates with NVIDIA for Stronger Agent Skill Security*. OpenClaw Blog, June 1, 2026.

[4] J. Merhi. *Where OpenClaw Security Is Heading*. OpenClaw Blog, May 15, 2026.

[5] J. P. Anderson. *Computer Security Technology Planning Study*. ESD-TR-73-51, USAF Electronic Systems Division, 1972.

[6] J. H. Saltzer, M. D. Schroeder. *The Protection of Information in Computer Systems*. Proceedings of the IEEE 63(9), 1975.

[7] H. Inan et al. *Llama Guard: LLM-based Input-Output Safeguard for Human-AI Conversations*. arXiv:2312.06674, 2023.

[8] T. Rebedea et al. *NeMo Guardrails: A Toolkit for Controllable and Safe LLM Applications with Programmable Rails*. EMNLP 2023 System Demonstrations, arXiv:2310.10501.

[9] S. Saha, F. Faghih, S. Feizi. *Under the Hood of SKILL.md: Semantic Supply-chain Attacks on AI Agent Skill Registry*. arXiv:2605.11418, 2026.

[10] *Securing Agentic AI: From Per-Action Checks to Trajectory Assurance*. arXiv:2608.01558, 2026.

[11] *Enforcing Temporal Constraints for LLM Agents*. arXiv:2512.23738, 2026.